楕円曲線暗号

実装を通して学ぼう

wuhu1sland (49th)

暗号の分類

共通鍵暗号

AES
使用例: Wi-Fi, SSH (暗号化)

ChaCha20
使用例: WireGuard (暗号化)

公開鍵暗号

RSA

ECC (楕円曲線暗号)

使用例: HTTPS, SSH, VPN (署名・鍵交換など)

共通鍵暗号は主に「暗号化」、公開鍵暗号は主に「認証」に使用される

署名と鍵交換

デジタル署名

送信者が秘密鍵でメッセージに署名

受信者が公開鍵で署名を検証

→ 送信者の本人確認 + 改ざん検出

鍵交換 (鍵共有)

事前の秘密の共有なしに

盗聴の可能性のある通信路を使って

→ 暗号鍵の共有を可能にする

RSA と ECC の最大の違い — 安全性の根拠

RSA

素因数分解の困難性

大きな数を素因数分解するのは計算量的に非常に困難

ECC

離散対数問題の困難性

$g^x \equiv m \pmod p$ で m は計算可能だが
g, p, m から x (離散対数) を求めるのが困難

ECCの方が短い鍵長で同等の安全性を実現できる

ECC (楕円曲線暗号) とは

楕円曲線上の離散対数問題の困難性を利用した公開鍵暗号方式

ECDH	Elliptic curve Diffie-Hellman key exchange	鍵交換
ECDSA	Elliptic Curve Digital Signature Algorithm	署名
Schnorr署名	1989年発明、2008年まで特許制限	署名
EdDSA	Edwards-curve Digital Signature Algorithm	署名

楕円曲線と演算の定義

Weierstrass 標準形

$$y^2 = x^3 + ax + b$$

ECDSA, Schnorr等で使用

Edwards 曲線

$$ax^2 + y^2 = 1 + dx^2y^2$$

EdDSAで使用

パラメータの値はアルゴリズムごとに異なり、曲線が非特異であることが必要

代表的な曲線例

secp256k1

Bitcoinで使用 (Weierstrass形式)
a = 0, b = 7

$$y^2 = x^3 + 7$$

Ed25519

ssh-keygenで使用 (Edwards形式)
a = -1, d = -121665/121666

$$-x^2 + y^2 = 1 - (121665/121666)x^2y^2$$

曲線の係数に加え、有限体の法 p、ベースポイント G、G の位数 n もパラメータとなる

楕円曲線上の演算と群構造

2点 P, Q を通る直線が曲線と交わる第3の点を x 軸反転した点が P + Q

単位元	無限遠点 O
逆元	x 軸対称の点 −P
結合法則	(P + Q) + R = P + (Q + R)

この演算により楕円曲線は群 (Group) となる

有限体上の楕円曲線

素数 p を法とする剰余集合 {0, 1, ..., p-1} = 有限体 F

有限体上で同じ演算を定義 → 曲線は群になる

ただし「曲線」には見えず、ランダムに見える点の集合となる

安全性の根拠

順に g を繰り返し加算すれば $g^x \equiv m$ は計算できるが、m と g から x を復元できない

ECDH — 楕円曲線 Diffie-Hellman 鍵交換

0 Alice・Bob 間で曲線パラメータ (G, n) を共有

1 各自ランダムな秘密鍵 dA, dB を生成し
QA = dA·G, QB = dB·G の公開鍵を交換

2 Alice: dA·QB / Bob: dB·QA
→ 同じ値 K = dA·dB·G を得る

秘密鍵を明かさず共通の値 K を共有 / 第三者は K を計算できない

ECDSA — 署名アルゴリズム

SSL証明書やBitcoinの署名など幅広く使われるスタンダード(1999年NIST標準化)

Alice (署名者)

1. 秘密鍵 dA → 公開鍵 QA = dA·G
2. e = H(m) → 上位ビットマスク z
3. 乱数 k → (x,y) = k·G
4. r = x mod n
   s = k^{-1}(z + r·dA) mod n
5. **署名 (r, s) を m と共に送信**

Bob (検証者)

1. (r, s) を受け取る
2. e = H(m) → z を計算
3. $u_1 = z \cdot s^{-1} \bmod n$
   $u_2 = r \cdot s^{-1} \bmod n$
4. $(x', y') = u_1G + u_2QA$
5. **$x' \equiv r \pmod n$ なら正当**

余談: PS3 事件 (2010)

SONY が全署名で同じ nonce k を使い回した結果、秘密鍵 dA が逆算された

同じ k → 同じ r / 2つのメッセージの署名から連立方程式を構成

$$k \equiv (z_1 - z_2) / (s_1 - s_2) \mod n$$
$$dA = (s \cdot k - z) / r \mod n$$

fail0verflow (marcan, bushing, sven) が nonce を特定
George Hotz が署名用秘密鍵を割り出し → jailbreak 可能に

余談: Snowden 事件 — Dual_EC_DRBG

NSA推奨の乱数生成器 Dual_EC_DRBG にバックドア疑惑

ベースポイントの決定根拠が不明瞭 →
NSA が事前にスカラー値を把握していた可能性

乱数生成パターンが予測可能になると nonce が割れ、秘密鍵復元のリスク

教訓: 暗号アルゴリズム自体だけでなく、乱数生成器の安全性も極めて重要

Schnorr 署名

1989年発明 / 2008年まで特許制限 / ECDSAより検証効率が良く、集約署名が可能

Alice (署名者)

1. 秘密鍵 dA / 公開鍵 QA = dA·G
2. 乱数 r → R = r·G
3. R || m を結合
4. s = r + H(R||m)·dA
5. **署名 (R, s) を送信**

Bob (検証者)

1. (R, s) を受け取る
2. 以下を確認:

$$s \cdot G = R + H(R||m) \cdot QA$$

めちゃくちゃシンプル。証明も簡単。

EdDSA — Edwards-curve Digital Signature Algorithm

Twisted Edwards 曲線を使用 (Weierstrass形式ではない)

Alice (署名者)

1. H(dA) 前半32B → s / QA = s·G
2. H(dA) 後半32B → prefix
3. r = H(prefix||m) mod n
   R = r·G
4. k = H(R||QA||m) mod n
5. S = (r + k·s) mod n
6. **署名 (R, S) を送信**

Bob (検証者)

1. (R, S) を受け取る
2. k = H(R||QA||m) mod n

3. 以下を確認:

$$8(S \cdot G) = 8(R + k \cdot QA)$$

EdDSA の特徴 — Schnorr 署名との違い

決定論的 nonce

nonce r を秘密鍵のハッシュから決定論的に計算 → 乱数生成器の安全性に依存しない

Cofactor 8 の存在

twisted Edwards曲線のベースポイントGの位数 n = 8L (素数ではない)

完全加法公式が使用可能 → 点の場合分け不要 → サイドチャネル攻撃耐性

小位数部分群の影響をキャンセルするため検証時に ×8 する

応用例 1: SSH

1. 鍵交換で共有秘密を作成 → 暗号化用の共通鍵を生成
2. サーバはホスト鍵で自己証明
   (クライアントは ~/.ssh/known_hosts と照合)
3. サーバがチャレンジを送信
   クライアントが秘密鍵で署名して返す
4. 公開鍵で検証 → 認証完了

応用例 2: Bitcoin

1. UTXOアンロック — 秘密鍵で未使用送金への署名を作成
2. トランザクション作成 — 署名+公開鍵をinput、送信先hashをoutput
3. 検証 — マイナーが公開鍵hashと署名を検証
4. PoW — nonce探索でブロック生成条件を満たすハッシュを発見
5. ブロックチェーン追加 — 他ノードが検証して各自追加

署名アルゴリズム: secp256k1 曲線の ECDSA

Bitcoin Taproot アップグレード (2021)

主な変更点

✗ 公開鍵のハッシュを output に記録	→	✓ **公開鍵の x 座標を output に記録**
✗ ECDSA 署名	→	✓ **Schnorr 署名**

メリット

Schnorr署名の線形性 → マルチシグ / バッチ検証 / パフォーマンス向上
key-path spend に加え、マークルツリーによる script-path spend も実装

演習

EdDSA / Schnorr署名 / DH鍵交換 を実装してみよう